# TGT: Temporal Gait Transformer for Analyzing Biomechanics of Walking Abnormalities

Sebastian Kot*
City, University of London

Mohammed Kairmi*
City, University of London

*Equal contribution

## Abstract

We present the Temporal Gait Transformer (TGT), a lightweight CNN-Transformer model that classifies five walking activities (normal walking, injury walking, stepping, swaying, and jumping) from raw smart insole data. Each insole records 18 pressure values, 3 accelerometer axes, and 3 Euler orientation angles (roll, pitch, yaw), giving 24 channels per foot and 48 features per timestep after merging left and right streams at 60 Hz. We segment each recording into overlapping windows of 80 timesteps (1.33 s) with a stride of 30 and z-score normalise every window independently to remove subject-specific baselines. A two-layer 1D convolutional stem (32 channels) extracts local gait patterns such as heel-strike and toe-off phases, then two Transformer encoder layers with two-head self-attention capture longer-range dynamics across the gait cycle. Timestep representations are aggregated by mean pooling rather than a learnable classification token, which proved more stable with limited training subjects. Training uses label-smoothed weighted cross-entropy, a weighted random sampler to oversample minority classes, and five augmentation transforms (Gaussian jitter, per-channel scaling, channel dropout, temporal masking, and random time shifts). All splits are performed at the file level to prevent data leakage from overlapping windows.

## 1 Data Preprocessing

### 1.1 Data Types

Data is collected from smart insoles worn on both feet. At every timestep each insole records pressure at 18 locations, acceleration along 3 axes, gyroscope readings along 3 axes, magnetometer readings along 3 axes, a 4-component orientation quaternion, derived Euler angles (roll, pitch, yaw), and an insole identifier (left or right). Additional fields such as sub-device timestamps and battery status are also present but not used. A full list of raw fields is given in Table 4 in the Appendix.

### 1.2 Separating and Merging Input Data

Each CSV file contains interleaved rows from the left and right insoles. We split them using the insole identifier field (`ele_36`: 1 for left, 0 for right). From each side we keep 24 channels: the 18 pressure sensors (`ele_0` to `ele_17`), the 3 accelerometer axes (`ele_18`, `ele_19`, `ele_20`), and the 3 Euler orientation angles (`ele_22` = roll, `ele_23` = pitch, `ele_24` = yaw). Gyroscope, magnetometer, and quaternion fields are dropped because the Euler angles already capture foot orientation in a more compact form.

We then align left and right rows by timestamp using a nearest-match merge with a tolerance of $\delta t < 0.02$ s. Row pairs that exceed this tolerance are discarded. The result is a single table where each row describes both feet simultaneously, with 48 features in total (24 per foot). The retained channels are listed in Table 5 in the Appendix.

### 1.3 Sampling Rate

Different recordings arrive at different native frame rates depending on device clock drift and Bluetooth throughput. We resample every merged recording to a fixed 60 Hz. Streams below 60 Hz are upsampled by index-based linear interpolation; streams above 60 Hz are

downsampled by selecting evenly spaced indices. This guarantees that one row always represents exactly 1/60 s regardless of the original recording conditions.

**Note on upsampling correction.** An earlier version of the upsampling routine generated evenly spaced target timestamps and called `reindex` directly on the original data frame. Because target timestamps almost never coincided exactly with recorded timestamps (floating-point mismatch on ten-digit Unix epochs), the operation discarded virtually all original samples and left a single anchor point, causing `interpolate` to produce constant rows. This affected 35 of 92 Injury_walking training files and all four Group 10 recordings whose native frame rate fell below 60 Hz. The corrected implementation first takes the union of original and target timestamp indices, interpolates over the combined index so every original measurement serves as an anchor, and then selects the target timestamps.

### 1.4 Windowing

We segment each recording into fixed-length windows of $W = 80$ consecutive timesteps (1.33 s at 60 Hz), enough to cover roughly one full gait cycle. Consecutive windows are spaced by a stride of $S = 30$, giving 62.5% overlap. The overlap increases the number of training samples and ensures that gait events near window boundaries still appear in full within at least one window. Each window is a matrix of shape $80 \times 48$.

### 1.5 Normalisation

We normalise the data in two stages.

**Global z-score.** We compute the mean and standard deviation of each of the 48 features across all training windows and use these statistics to standardise every window (training, validation, and evaluation). Fitting statistics on training data only prevents information leakage. This step brings features such as pressure and acceleration onto a comparable numerical scale.

**Per-window z-score.** After the global step, each window $\mathbf{X} \in \mathbb{R}^{W \times F}$ is independently normalised along the time axis:

$$\hat{x}_{t,f} = \frac{x_{t,f} - \mu_f}{\sigma_f + \varepsilon}, \qquad \varepsilon = 10^{-8} \tag{1}$$

where $\mu_f$ and $\sigma_f$ are the mean and standard deviation of feature $f$ across the $W$ timesteps of that window. This removes subject-specific absolute baselines (resting pressure levels, accelerometer offsets from body weight or sensor placement) so the model learns from the temporal *shape* of signals rather than their magnitude.

### 1.6 Labels

Each window inherits the activity label of the folder it came from. The five classes are listed in Table 1:

### 1.7 Train/Validation Split

Overlapping sliding windows from the same recording share most of their content. A naive random split would place near-identical windows into both training and validation sets, artificially inflating validation scores. We avoid this by splitting at the *file* level: all windows extracted from a single CSV stay in the same partition.

We use a 90/10 train/validation split over files. The validation set is used solely to monitor generalisation and select the best model checkpoint via early stopping. All available recordings, including Group 10 data, are included in the pool to maximise subject diversity. Final evaluation is performed on a completely separate set of recordings that are not seen during training.

## 2 Methodology

The TGT is a lightweight hybrid CNN-Transformer model. A convolutional stem first extracts local temporal features from the raw sensor window, then a Transformer encoder models longer-range dependencies across the gait cycle. Timestep representations are aggregated by mean pooling, and a linear head produces class logits. The full pipeline is illustrated by the stages below.

## 2.1 1D Convolutional Stem

The model receives a window of shape $W \times 48$ ($W = 80$ timesteps, 48 sensor features each). We transpose to $48 \times W$ and apply two 1D convolutions along the time axis:

- Layer 1: kernel size 7, maps 48 input channels to $d_{\text{model}} = 32$ channels.
- Layer 2: kernel size 5, maps 32 to 32 channels.

Both layers use same-padding so the temporal length stays at $W$. Each convolution is followed by batch normalisation, a GELU activation, and dropout (0.3).

Gait signals contain short-range temporal patterns (the pressure wave from heel-strike to toe-off, the acceleration spike at foot contact) that are best captured by small convolutional filters. These local features are more consistent across subjects than raw sensor magnitudes, which vary with body weight, sensor placement, and foot shape. After the stem, we transpose back to $W \times 32$ for the Transformer.

## 2.2 Positional Encoding

Since the Transformer has no built-in notion of ordering, we add a fixed positional encoding to each token so the model can distinguish early timesteps from later ones within the window. For a given position $\text{pos} \in \{0, 1, \ldots, W-1\}$ and a dimension index $i \in \{0, 1, \ldots, 15\}$ we compute:

$$PE_{\text{pos},\,2i} = \sin\left(\frac{\text{pos}}{10000^{\,2i/d_{\text{model}}}}\right) \tag{2}$$

$$PE_{\text{pos},\,2i+1} = \cos\left(\frac{\text{pos}}{10000^{\,2i/d_{\text{model}}}}\right) \tag{3}$$

with $d_{\text{model}} = 32$. Lower dimensions oscillate rapidly and capture fine-grained local timing, while higher dimensions oscillate slowly and encode where the timestep sits within the segment. The result is a $80 \times 32$ matrix of positional vectors added element-wise to the convolutional features.

## 2.3 Transformer Encoder

The core of the model is a stack of $N = 2$ identical Transformer encoder layers. Each layer consists of two sub-blocks.

### 2.3.1 Multi-Head Self-Attention

For each of the 80 tokens, the layer computes Query ($Q$), Key ($K$), and Value ($V$) vectors via learned linear projections. The attention score between any two tokens is the scaled dot product of one token's query with another's key. These scores form an $80 \times 80$ matrix, passed through a softmax so that each row sums to one. Multiplying by the Value matrix produces a new representation where each token is a weighted combination of all tokens, according to their relevance to it.

We use $H = 2$ attention heads in parallel. Each head operates on a $32/2 = 16$ dimensional slice of the embedding, and the results are concatenated back to 32 dimensions.

### 2.3.2 Feed-Forward Network

After attention, each token passes independently through a two-layer feed-forward network (FFN). The FFN projects from 32 to 128 dimensions (a $4\times$ expansion), applies a GELU activation, and projects back to 32 dimensions. Residual connections and layer normalisation are applied around both the attention sub-block and the FFN sub-block.

A dropout rate of 0.3 is applied inside both the attention and FFN sub-blocks. The two encoder layers are applied sequentially, with each one refining the token representations further.

## 2.4 Mean Pooling and Classification Head

After the final encoder layer, we compute the mean of all 80 token representations along the time axis, producing a single 32-dimensional vector. We chose mean pooling over a learnable [CLS] token because, with training data from only a small number of subjects, the CLS token tended to overfit to subject-specific patterns rather than learning a useful aggregate representation. Mean pooling is parameter-free and gives a stable summary regardless of training set size.

This pooled vector is passed through layer normalisation, dropout, and a fully connected linear layer that outputs 5 values, one per class. A softmax is applied to these logits to obtain a probability distribution over the five activities.

### 2.5 Training Configuration

We use cross-entropy loss with label smoothing ($\alpha = 0.1$). Label smoothing softens the target distribution from a hard one-hot vector to a mixture that reserves a small probability for incorrect classes, preventing the model from becoming overconfident when the number of distinct training subjects is small.

The training set is class-imbalanced: Swaying has roughly twice as many windows as Stepping. We handle this at two levels. First, the loss is weighted by inverse class frequency so that errors on rare classes produce proportionally larger gradients. Second, a **weighted random sampler** draws windows with probability inversely proportional to class frequency, so every class is seen roughly equally often per epoch. Together these ensure that minority classes like Stepping receive enough training signal despite having fewer recording sessions.

We optimise with AdamW (decoupled weight decay). Gradients are clipped to a maximum norm of 1.0 per step to prevent occasional large updates from destabilising training. All training parameters are listed in Table 2.

The learning rate starts at $5 \times 10^{-4}$ and follows a cosine decay schedule over the full 150 epochs. If validation accuracy does not improve for 80 consecutive epochs, training stops early. The checkpoint with the highest validation accuracy is saved and used for all subsequent evaluation.

### 2.6 Data Augmentation

Training data comes from a small number of subjects, so the model risks memorising subject-specific patterns. We apply five augmentation transforms to each training window on every epoch to simulate the variation expected from unseen subjects:

1. **Gaussian jitter.** Additive noise sampled from $\mathcal{N}(0, 0.15^2)$ is added independently to every element of the window. This simulates sensor noise variation between recording sessions and devices.

2. **Per-channel amplitude scaling.** Each of the 48 features is multiplied by an independent random factor drawn uniformly from $[0.5, 1.5]$. This forces the model to tolerate differences in sensor calibration and body weight across subjects.

3. **Channel dropout.** Each feature channel is independently zeroed out with probability 0.2. This prevents the model from relying too heavily on any single sensor and improves robustness to occasional sensor failures.

4. **Temporal masking.** A contiguous block of up to $\lfloor W/7 \rfloor$ timesteps (roughly 15% of the window) is randomly selected and set to zero, acting as a regulariser that forces the model to use the full temporal context.

5. **Random time shift.** The window is circularly shifted along the time axis by a random offset of up to $\pm W/8$ timesteps. This makes the model invariant to where exactly within the recording the gait event starts.

Augmentation is applied online during training only; validation and evaluation data are not augmented.

In addition to these per-window transforms, we apply **mixup** at the batch level during training. For each mini-batch, we sample a mixing coefficient $\lambda \sim \text{Beta}(\alpha, \alpha)$ with $\alpha = 0.2$ and blend each input window with a randomly paired window from the same batch: $\tilde{x} = \lambda\, x_i + (1 - \lambda)\, x_j$. The loss is computed as $\lambda\, \mathcal{L}(f(\tilde{x}),\, y_i) + (1 - \lambda)\, \mathcal{L}(f(\tilde{x}),\, y_j)$. With $\alpha = 0.2$ the mixing is usually mild (most $\lambda$ values are close to 0 or 1), but it still smooths decision boundaries and reduces the tendency to memorise individual training windows.

### 2.7 Model Summary

Table 3 summarises the architectural choices of the TGT.

## 3 Evaluation and Results

*To be completed after training runs.*

# 4 Design Challenges and Reasoning

## 4.1 Data Leakage from Window-Level Splitting

An early version of the pipeline split data at the window level: windows were randomly shuffled into train, validation, and test sets. Because consecutive windows overlap (stride $< W$), near-identical windows ended up on both sides of the split. Validation accuracy appeared high (>87%) but performance on genuinely unseen recordings was around 37%.

We solved this with file-level splitting: all windows from a single CSV recording stay in the same partition. The same leakage problem was present in the baseline MLP notebooks, meaning their reported evaluation accuracy was similarly inflated.

## 4.2 Limited Subject Diversity

The training set contains recordings from only a handful of subjects. Without mitigation, the model memorises subject-specific quirks (resting pressure levels, body weight effects, sensor placement) rather than learning generalisable gait patterns. We addressed this with four complementary strategies:

- Including Group 10 recordings in the training pool to increase subject diversity.
- Per-window z-score normalisation, which strips away absolute sensor baselines so the model sees only temporal shape.
- Aggressive augmentation (five transforms) that simulates the variation expected from new subjects.
- A deliberately small model (32-dimensional embeddings, ∼15k parameters) that has limited capacity to memorise.

## 4.3 Class Imbalance

Stepping has far fewer recording sessions than Swaying or Injury_walking. Without correction the model learns to ignore it, since overall accuracy stays high by getting majority classes right.

We address this at two levels: inverse-frequency loss weighting (larger gradients for rare-class errors) and a weighted random sampler that oversamples minority classes so every class appears roughly equally often per epoch. Despite both fixes, Stepping remains the weakest class (F1 around 0.28–0.35), suggesting it may be genuinely ambiguous from insole pressure and orientation data alone.

## 4.4 Architecture: CNN-Transformer Hybrid

An earlier version used a pure Transformer with a per-timestep linear projection and a learnable CLS token. The linear projection treats each timestep independently and misses local temporal patterns (heel-strike pressure waves, acceleration spikes). The CLS token, a single learnable vector, overfitted quickly with so few training subjects.

Replacing the linear projection with a 1D CNN stem lets the model extract local temporal features before the Transformer processes them. Replacing the CLS token with mean pooling removes a learnable component and produces a more stable summary with limited data.

## 4.5 Model Size

The baseline MLP uses roughly 800k parameters. With few training subjects, a large model can memorise individual subjects rather than learn transferable patterns. We deliberately kept the TGT small ($d_{\text{model}} = 32$, 2 heads, 2 encoder layers, 128 FFN hidden dimension, ∼15k total parameters) to limit memorisation capacity and force compact representations.

## 4.6 Dual Normalisation

A global z-score (fit on training data only) brings all 48 features onto a comparable scale. A subsequent per-window z-score removes the absolute offset within each window so the model learns from relative temporal patterns rather than subject-specific magnitudes. This two-stage approach targets the main source of cross-subject variation: baseline sensor readings (resting pressure, standing orientation) differ between individuals, while the temporal dynamics of each activity are more consistent.

### 4.7 Mixup Augmentation

We experimented with mixup ($\alpha = 0.2$), which blends pairs of training windows and their labels to smooth decision boundaries. Training accuracy drops to around 55–65% under mixup because accuracy is measured against hard labels while training targets are blended. Validation accuracy improved from 91.3% to 92.0%. However, without cross-validation the 0.7 percentage point gain is within noise, so we treat mixup as optional.

### 4.8 Training Stability

Gradient clipping (max norm 1.0) prevents occasional large gradients from destabilising training, which otherwise produces bumpy loss curves. Label smoothing (0.1) acts as a soft regulariser by preventing overconfident predictions. Cosine annealing with generous patience (80 epochs) gives the model enough time to converge under heavy augmentation, which slows learning. Early stopping on validation accuracy ensures we keep the best-generalising checkpoint.

### 4.9 Comparison with Baseline MLP

The baseline MLP notebooks report high evaluation accuracy, but their evaluation windows come from the same recording sessions as training windows (often overlapping by >50% due to window-level splitting). The TGT with file-level splitting evaluates on genuinely held-out recordings. Its reported numbers are lower but reflect real generalisation performance.

| Class ID | Label | Description |
|---|---|---|
| 0 | Normal_walking | Baseline gait pattern without perturbations or injury. Used as reference for distinguishing altered gait behaviours. |
| 1 | Injury_walking | Walking while simulating or experiencing injury (e.g., limping, reduced load on one side), leading to asymmetric pressure and motion patterns. |
| 2 | Stepping | Repeated stepping in place or step-like movements with frequent foot lifts and contacts but limited forward progression. |
| 3 | Swaying | Predominantly static stance with body sway, where the centre of pressure moves but feet largely remain planted. |
| 4 | Jumping | Dynamic movements with clear take-off and landing phases, characterised by brief loss of contact and high impact forces on landing. |

Table 1: Activity classes used for supervised learning. Each 0.33 s window is assigned one of five class IDs according to the folder name of the source recording, following the mapping implemented in the code.

# Appendix

## Insole Feature Definitions

## Retained Input Features

| Parameter | Value |
|---|---|
| Optimiser | AdamW |
| Learning rate | $5 \times 10^{-4}$ |
| Weight decay | $1 \times 10^{-3}$ |
| LR schedule | Cosine annealing |
| Batch size | 64 |
| Max epochs | 150 |
| Early stopping patience | 80 epochs |
| Dropout | 0.3 |
| Label smoothing | 0.1 |
| Gradient clipping | Max norm 1.0 |
| Loss weighting | Inverse class frequency |
| Training sampler | Weighted random (oversample minority) |
| Mixup $\alpha$ | 0.2 |

Table 2: Training hyperparameters used for TGT.

| Component | Value |
|---|---|
| Input features per timestep | 48 |
| Window size ($W$) | 80 |
| Stride ($S$) | 30 |
| Conv stem | 2 layers (kernel 7, kernel 5) |
| Channel dimension ($d_{\text{model}}$) | 32 |
| Attention heads ($H$) | 2 |
| Encoder layers ($N$) | 2 |
| FFN hidden dimension | 128 |
| Positional encoding | Fixed sinusoidal |
| Sequence aggregation | Mean pooling |
| Per-window normalisation | z-score per feature |
| Global normalisation | z-score (fit on train) |
| Data augmentation | Jitter, scaling, channel dropout, time mask, time shift |
| Output classes | 5 |

Table 3: TGT model architecture summary.

| Signal | Range | Components | Elements | Notes |
|---|---|---|---|---|
| Cap sensing (pressure) | $0 \ldots 800$ | 18 electrodes | Ele_0–Ele_17 | Per-electrode plantar pressure. |
| Accelerometer (x, y, z) | $-1 \ldots 1$ | $x, y, z$ | Ele_18–Ele_20 | 3D linear acceleration. |
| Accelerometer (time) | $> 0$ | timestamp | Ele_21 | **Redundant:** inner accelerometer timestamp (same timestep as row, dropped in preprocessing). |
| Gyroscope (roll, pitch, yaw) | $-600 \ldots 600$ | $r, p, y$ | Ele_22–Ele_24 | Angular velocity in °/s. |
| Gyroscope (time) | $> 0$ | timestamp | Ele_25 | **Redundant:** inner gyroscope timestamp (same timestep as row, dropped in preprocessing). |
| Magnetometer (x, y, z) | $-3000 \ldots 3000$ | $x, y, z$ | Ele_26–Ele_28 | Local magnetic field. |
| Magnetometer (time) | $> 0$ | timestamp | Ele_29 | **Redundant:** inner magnetometer timestamp (same timestep as row, dropped in preprocessing). |
| Quaternion | $-1 \ldots 1$ | $x, y, z, v$ | Ele_30–Ele_33 | Orientation computed from all previous sensors. |
| Quaternion (time) | $> 0$ | timestamp | Ele_34 | **Redundant:** inner quaternion timestamp (same timestep as row, dropped in preprocessing). |
| Unused | — | — | Ele_35 | Not used (reserved). |
| Insole side label | $\{0, 1\}$ | right / left | Ele_36 | Right = 0, Left = 1. |
| Battery status | $0 \ldots 100$ | percentage | Ele_37 | **Redundant:** remaining battery level (not used in the model). |

Table 4: Overview of per-timestep insole features, their numeric ranges, and element indices in the raw data vector.

| Signal | Range | Components | Elements | Rationale for inclusion |
|---|---|---|---|---|
| Cap sensing (pressure) | $0 \ldots 800$ | 18 electrodes | Ele_0–Ele_17 | High-resolution plantar pressure map. Captures stance phase, load distribution, and contact pattern differences across activities (e.g., forefoot vs. heel loading in jumping vs. walking). |
| Accelerometer (linear) | $-1 \ldots 1$ | $x, y, z$ | Ele_18–Ele_20 | Foot-frame linear acceleration. Encodes impact magnitude, swing dynamics, and vertical motion, which are highly discriminative for gait phase transitions and dynamic activities such as jumping. |
| Quaternion (orientation) | $-1 \ldots 1$ | $x, y, z, v$ | Ele_30–Ele_33 | 3D orientation of the insole, fused from inertial and magnetic sensors. Provides a numerically stable, gimbal-lock-free representation of foot pose over time, enabling the model to learn differences in foot rotation patterns between gait types. |

Table 5: Subset of insole signals retained for model input. We keep per-timestep plantar pressure, linear acceleration, and fused orientation (quaternions) for each foot, as these jointly capture contact mechanics, dynamic motion, and foot pose. Redundant timestamps, raw gyroscope/magnetometer channels, and battery state are excluded from the learning pipeline.